iscte INSTITUTO UNIVERSITÁRIO DE LISBOA

Escola de Tecnologia e Arquitetura

Projeto de Inteligência Artificial

2023/2024

Docente: *Professor Sancho Oliveira*

Professora Rita Peixoto

Professora Mariana Rodrigues

Professora Filipa Costa

Autoria:

Miguel van Raamsdonk nº110722, *Manuel Almeida* nº 111176

NN.java

O *design* da nossa rede neuronal é uma rede com 7 nós de input, 60 nós na camada intermédia cuja função de ativação é a sigmoide e finalmente 2 nós de output. É de notar que os *hidden nodes* podem ser alterados consoante o tipo de teste efetuado.

Decidiu-se adicionar dois atributos (*values e fitness)* para guardar o valor de *fitness* da rede específica e os valores dos pesos da rede.

Na concepção da rede, utilizaram-se matrizes para armazenar os pesos e vetores para as *biases* que são inicializados com valores aleatórios.

O método *forward(int[] inputValues)* calcula o vetor *outputLayer* através do método tradicional e lecionado nas aulas.

O método *nextMove(int[] currentState)* escolhe o valor final da rede, através da "tradução" para um valor simbólico (1,2), comparando os dois *outputs.*

Adicionaram-se ainda métodos auxiliares.

GeneticAlgorithm.java

No começo da implementação deste algoritmo, definiram-se os atributos essenciais.

No construtor é onde o algoritmo é desenvolvido, e como habitual, inicializou-se as variáveis.

O método *genFirstPop()* permite gerar a primeira população de redes neuronais com pesos compreendidos entre –1 e 1; o resultado do mesmo é atribuído a *this.population*.

São inicializadas as variáveis r,j,t que são usadas para controlar a perfomance do algoritmo em *keepTrack().*

É iniciado o ciclo que percorre as gerações. De seguida é feita uma avaliação do desempenho de cada indivíduo da população através de um ciclo *for,* onde se atribui um valor *fitness* a cada um, e guarda-se o melhor indivíduo em *best.*

Com os valores *fitness* já atribuídos, a população é ordenada de acordo com esse parâmetro, e são escolhidos os 100 melhores indivíduos em *filter().*

É criada uma nova população, resultado do método *evolve().* Este método começa por selecionar os 5 melhores indivíduos da população atual para a seguinte. São selecionados aleatoriamente uma percentagem dos melhores indivíduos da população atual para serem os pais. São aplicados a esses pais os operadores genéticos *mutate() e reproduce()* (crossOver). Os filhos gerados são todos guardados em *newPop.*

Evolução

Nos resultados da evolução observou-se que havia uma evolução lenta e pouco eficiente. Por vezes, ao testar com 300 gerações a evolução inicial era bastante acentuada, mas estagnava no resto das centenas de gerações.

Um problema recorrente que se teve foi numa *seed* termos um melhor indivíduo com uma certa *fitness*, mas no jogo em si a *fitness* era sempre menor. Portanto, a bola acertava sempre uma vez a menos do que devia.

Para tentar evoluir mais eficientemente o algoritmo experimentou-se diversas técnicas de mutação e crossover, tais como: *one-point* e *two-point crossover*, *elitism*, somar ao peso um valor aleatório entre **α** e **β** e *swapping*. Tal como aumentar o nº de gerações para avaliar melhor a evolução e finalmente as diversificar as taxas/probabilidades de mutação, seleção e de *crossover*.

Outra tentativa que se fez foi na função de *getFitness()* desenvolvida pelos professores experimentar com *–time, +time* ou nem sequer incluir o *time*. Os resultados foram todos bastantes parecidos.
De forma a controlar a perfomance do algoritmo, foi criado o método *keepTrack(),* que de X em X gerações, caso não houvesse evolução, alterava a percentagem de mutação.
Todas estas técnicas foram implementadas com o objetivo de explorar ao máximo o espaço de procura.
Sobre o desempenho da rede neuronal pode-se observar que o método *forward* propaga a informação pelas camadas, o método *nextMove* vai pegar nessa informação, comparar os dois *outputs* e decidir se vai jogar para a esquerda ou para a direita. Foi possível observar o funcionamento da rede neuronal, visto que, a bola não ia sempre para o mesmo sítio. Logo, os valores de outputs iam variando e após comparar os dois o *nextMove* escolhia o maior valor.
A melhor *fitness* encontrada foi 799999 com *one-point* crossover e somar ao peso um valor aleatório entre **α(alpha)** e **β(beta)** com 100 gerações.

Outras aplicações

As redes neuronais são fundamentais quer seja para *Machine Learning* quer para reconhecimento de padrões, por exemplo. São utilizadas em tarefas supervisionadas como classificação. Outras finalidades, podem ser a compreensão da língua humana, tal como a sua tradução e escrita. Logo, as redes neuronais irão ser utilizadas no nosso dia a dia, pela importância e vasta utilidade que têm.
Um bom exemplo observa-se no mundo dos videojogos, pois as redes são usadas para controlar e determinar as ações dos *NPC* (*non-playable character*) que são uma parte integral de bastantes jogos. Também conseguem aprender, adaptar e mostrar que conseguem executar tarefas inteligentes. Na área da saúde podem ajudar com o diagnóstico médico e com a previsão dos resultados dos tratamentos.

Conclusão

Em suma, foi construído um algoritmo para jogar os jogos *Breakout* e *Pacman*. No caso do *Breakout*, apesar do resultado não ser ótimo (fitness não alcança a escala dos milhões), a rede neuronal consegue evoluir e aumentar a pontuação. Foram feitas várias tentativas para melhorar o algoritmo através da implementação de outras técnicas de operações genéticas, mas sem sucesso. Por outro lado, no *Pacman,* mudou-se ligeiramente o algoritmo e a rede neuronal, no entanto não se observa alguma evolução no jogo.
É de realçar que em ambos os jogos, depois de realizados testes, a fitness obtida pela melhor rede do algoritmo (observada através de *System.out.println()*) não correspondia à pontuação obtida no final quando é possível observar a jogabilidade.